LAPORAN PRAKTIKUM MIKROPROSESOR DAN INTERFACE

OPERASI PENGURANGAN BILANGAN OKTAL

Disusun untuk memenuhi tugas praktikum Mikroprosesor Dan Interface

Oleh

Gaitsha Azzahra Devitrya

231344012

Tanggal Percobaan: 21 Maret 2025

Tanggal Penyerahan: 25 April 2025

Instruktur: 1. Ferry Satria, BSEE., M.T.

2. Rahmawati Hasanah, S.ST., M.T.

PROGRAM STUDI D4 TEKNIK TELEKOMUNIKASI

JURUSAN TEKNIK ELEKTRO

POLITEKNIK NEGERI BANDUNG

2025

DAFTAR ISI

DAFTAR ISI ii

A. TUJUAN 2

B. LANDASAN TEORI 2

C. ALAT DAN BAHAN 5

D. LANGKAH KERJA 5

E. HASIL PERCOBAAN DAN ANALISIS DATA 6

G. KESIMPULAN 15

DAFTAR PUSTAKA 16

A. TUJUAN

1. Mahasiswa dapat memahami konsep operasi pengurangan bilangan oktal.
2. Mahasiswa dapat membuat program pada EMU8086 untuk menyelesaikan operasi penjumlahan bilangan oktal.

B. LANDASAN TEORI

A. Definisi microprosesor

Mikroprosesor dikenal juga sebagai CPU (Central Processing Unit). CPU terdiri dari ALU, Control Unit, Register , Bus System, dan Clock untuk melakukan komputasi.

B. Register

Register merupakan sebagian memori dari mikroprosesor yang dapat diakses dengan kecepatan yang sangat tinggi. Dalam melakukan pekerjaannya mikroprosesor selalu menggunakan register-register sebagai perantaranya, jadi register dapat diibaratkan sebagai kaki dan tangannya mikroprosesor.

Register merupakan sebagian memori dari mikroprosesor yang dapat diakses dengan kecepatan yang sangat tinggi. Dalam melakukan pekerjaannya mikroprosesor selalu menggunakan register-register sebagai perantaranya, jadi register dapat diibaratkan sebagai kaki dan tangannya mikroprosesor.

Terdapat dua jenis register yang dibedakan menurut besar Bit, yaitu Register 8-Bit dan Register 16-Bit.

Register 8-Bit hanya memiliki 1 bagian jenis register yaitu Register umum : AH, AL, BH, BL, CH, CL, DH, DL

Dan Register 16-Bit memiliki 5 bagian jenis register yang memiliki tugasnya masing-masing, yaitu :

1. Segmen Register.

   Register yang termasuk dalam kelompok ini terdiri atas register CS (Code Segment), DS (Data Segment), ES (Extra Segment) dan SS (Stack Segment) yang masing-masingnya merupakan register 16 bit. Register-register dalam kelompok ini secara umum digunakan untuk menunjukkan alamat dari suatu segmen.

2. General Register (Register Umum)

Register yang termasuk dalam kelompok ini adalah register AX, BX, CX dan DX yang masing-masing terdiri atas 16 bit. Register- register 16 bit dari kelompok ini mempunyai suatu ciri khas, yaitu dapat dipisah menjadi 2 bagian dimana masing-masing bagian terdiri atas 8 bit, Akhiran H menunjukkan High sedangkan akhiran L menunjukkan Low.

3. Register Pointer

Register ini menunjukkan alamat sebuah data di lokasi memori, dipakai saat operasi perpindahan data(dari/ke memori) dan penunjukkan alamat suatu instruksi. Jenis-jenis register pointer yaitu BP (Base Pointer) dan IP (Instruction Pointer).

4. Register Index

Sama dengan pointer register, sering digunakan untuk menunjukkan alamat sebuah data di lokasi memori pada operasi string. Jenisnya ada SI (Source Index), DI (Destination Index).

5. Register Stack

Register Stack (SP) atau Stack Pointer adalah salah satu jenis register dalam arsitektur komputer yang digunakan untuk menunjuk alamat teratas dari stack dalam memori.

6. Register Flag [REG 16 BIT]

Register Flag berfungsi untuk menunjukkan status(keadaan) sesaat dari mikroprosesor. Bit-bit pada flag akan mengalami perubahan, tergantung proses yang berlangsung. Sesuai dengan namanya Flags(Bendera) register ini menunjukkan kondisi dari suatu keadaan< ya atau tidak >. Karena setiap keadaan dapat digunakan 1 bit saja, maka sesuai dengan jumlah bitnya, Flags register ini mampu memcatat sampai 16 keadaan.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
-	-	-	-	OF	DF	IF	TF	SF	ZF	-	AF	-	PF	-	CF

C. Bahasa Assembly

Bahasa Assembly (rakitan) adalah bahasa pemrograman tingkat rendah yang digunakan pada perangkat yang dapat di program seperti komputer dan mikroprosesor. Instruksi-instruksi pada bahasa assembly menggantikan kode-kode biner dari bahasa mesin dengan mnemonic yang mudah diingat. Misal, instruksi penambahan dalam bahasa mesin memiliki kode "10110011" sedangkan dalam bahasa assembly dapat dibuat dalam instruksi mnemonic ADD, sehingga lebih mudah diapahami oleh manusia.

Bahasa assembly mempunyai hubungan 1-1 dengan berdasarkan instruksi mesin. Contohnya, untuk setiap julukan atau mnemonic yang nanti akan ditulis dengan menggunakan bahasa assembly, maka nanti juga akan langsung diterjemahkan dengan tepat satu kode operasi sehingga bisa langsung dimengerti oleh komputer.

D. Syntax/Perintah Mikroprosesor 8086

1. SUB

Perintah SUB akan mengurangkan nilai pada Tujuan dengan Asal. Hasil yang didapat akan ditaruh pada Tujuan, dalam bahasa pascal sama dengan instruksi Tujuan:=Tujuan-Asal. Untuk menolkan suatu register bisa menggunakan perintah seperti ini : SUB AX, AX. Syntax yang digunakan: SUB Tujuan, Asal.

2. SBB (Subtract with Borrow)

Seperti pada operasi penjumlahan, maka pada operasi pengurangan dengan bilangan yang besar(lebih dari 16 bit). Digunakan jika operasi pengurangan melibatkan carry dari operasi sebelumnya. Gunakan perintah SUB disertai dengan SBB (Substract With Borrow). Perintah SBB digunakan dengan syntax: SBB: Tujuan, Asal .Perintah SBB akan mengurangkan nilai Tujuan dengan Asal dengan cara yang sama seperti perintah SUB, kemudian hasil yang didapat dikurangi lagi dengan Carry Flag(Tujuan:=Tujuan-Asal-CF).

3. DEC (decrement)

Perintah DEC(Decrement) digunakan khusus untuk pengurangan dengan 1. Perintah DEC hanya menggunakan 1 byte memory, sedangkan perintah SUB dan SBB menggunakan 3 byte. Oleh sebab itu bila anda ingin melakukan operasi pengurangan dengan 1 gunakanlah perintah DEC. Syntax pemakaian

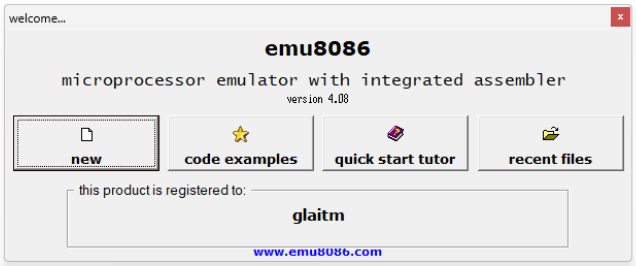
perintah dec ini adalah: DEC Tujuan Nilai pada tujuan akan dikurangi 1, seperti perintah Tujuan:=Tujuan-1 dalam Turbo Pascal. Tujuan disini dapat berupa suatu register maupun memory.

C. ALAT DAN BAHAN

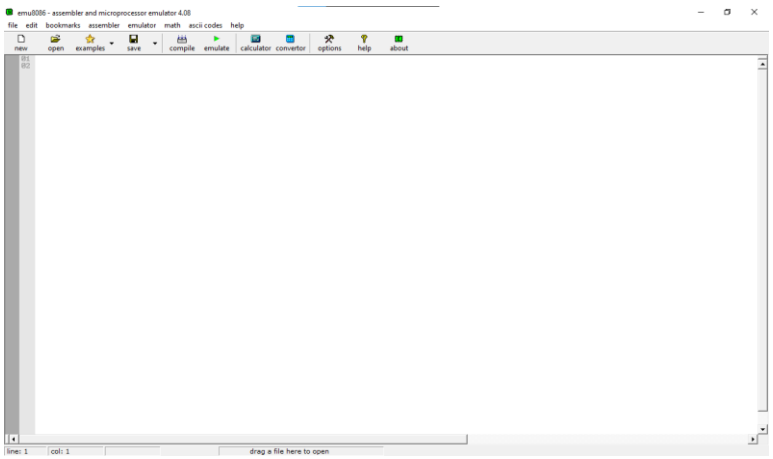
1. Laptop/PC
2. Software EMU 8086

D. LANGKAH KERJA

1. Instal Software emu8086 pada laptop dan buka aplikasinya
2. Setelah membuka aplikasi maka akan muncul tampilan *new* pada monitor



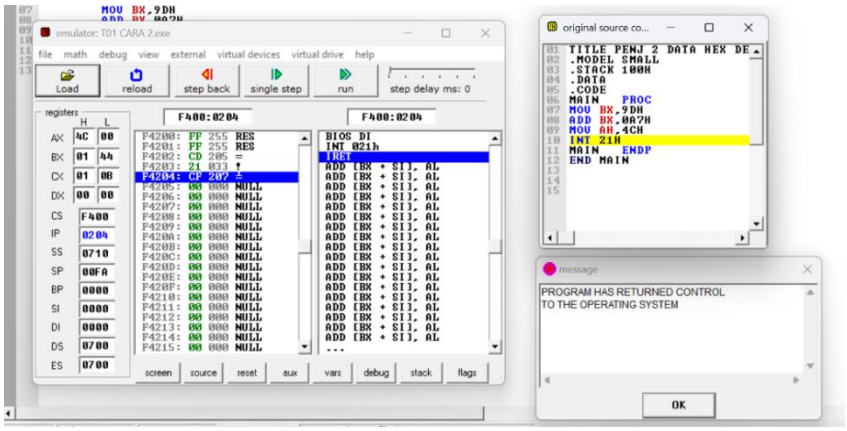
3. Maka setelah itu akan tampil worksheet seperti gambar dibawah ini :



Worksheet ini digunakan untuk menuliskan kode.

4. Untuk memeriksa kode yang berada di worksheet apakah program sudah benar atau belum, klik “compile”
5. Jika sudah disave maka jika program sudah benar akan ada tulisan “run”.

6. Jika sudah di ”run” maka akan muncul hasilnya pada monitor



E. HASIL PERCOBAAN DAN ANALISIS DATA

Pada praktikum ini, menggunakan aplikasi EMU8086 dimana praktikum ini mempelajari bagaimana cara membuat program untuk melakukan operasi bilangan oktal pada bahasa assembly.

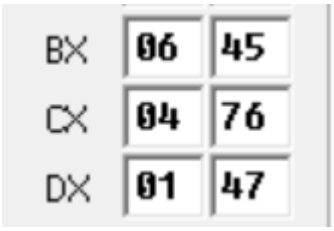
Pada praktikum kali ini terdapat 4 soal yang berhasil dikerjakan yang dapat dianalisis sebagai berikut :

1. Buat program untuk mengurangkan data oktal berikut : 645 – 476 = 147

Program :

```
TITLE PEGURANGAN 2 DATA OKTAL 3 DIGIT (SOAL1)
.MODEL SMALL
.STACK 100H
.DATA
.CODE
MAIN PROC
     MOV DX,BX
     SUB DX,CX
     MOV AL,DL
     AND AL,0FH
     CMP AL,8
     JC SANA
     SUB DX,8
SANA: MOV AL,DL
     CMP AL,80H
     JC SINI
     SUB DX,80H
SINI: MOV AL,DH
     AND AL,0FH
     CMP AL,8
     JC SONO
     SUB DX,800H
SONO: MOV AH,4CH
     INT 21H
MAIN ENDP
END MAIN
```

Hasil program :



Analisis data :

Pada soal ke-1, program dibuat untuk melakukan operasi pengurangan dua bilangan dalam format oktal dengan tiga digit. Program ini dibuat menggunakan software EMU 8086 dan menerapkan beberapa instruksi dasar seperti pemindahan data antar register, operasi logika, serta kondisi perulangan.

Dalam program ini, register BX dan CX menyimpan dua bilangan yang akan dikurangi. Instruksi MOV DX,BX dan SUB DX,CX digunakan untuk menyimpan hasil pengurangan ke dalam register DX. Setelah pengurangan dilakukan, program menggunakan operasi logika AND AL, 0FH serta perbandingan CMP AL,8 untuk mengecek apakah hasil pengurangan memerlukan koreksi dalam sistem bilangan oktal. Jika kondisi tertentu terpenuhi, program akan mengurangi nilai dalam register DX dengan angka 8 atau 800H untuk memastikan hasil tetap dalam format oktal.

Kemudian setelah program di running, hasil menunjukkan nilai register setelah operasi dilakukan. Dari gambar yang diberikan, nilai BX adalah 0645H, CX adalah 0476H, dan DX memiliki nilai 0147H setelah pengurangan. Ini menunjukkan bahwa program berhasil melakukan pengurangan dalam sistem oktal dengan memperhitungkan kemungkinan carry dan format bilangan oktal.

Secara keseluruhan, program ini menggunakan teknik manipulasi bit untuk memastikan hasil pengurangan tetap valid dalam sistem bilangan oktal. Program juga memanfaatkan instruksi percabangan JC untuk menangani kemungkinan nilai negatif dalam hasil pengurangan. Hal ini menunjukkan pemahaman yang baik tentang pengolahan bilangan dalam Assembly, khususnya dalam sistem oktal.

Pada program ini terdapat perintah yang baru dikenal dan digunakan dalam membuat program operasi pengurangan bilangan oktal, yaitu intruksi JC (Jump if

Carry) yang digunakan untuk berpindah ke suatu label tertentu jika kondisi Carry Flag (CF) = 1 setelah operasi pengurangan dilakukan. Pada program terdapat 3 penggunaan instruksi JC yaitu (JC SANA, JC SINI, JC SONO).

Pada instruksi JC SANA, perintah ini membandingkan nilai dalam register AL dengan 8. Jika nilai dalam AL lebih kecil dari 8, maka Carry Flag (CF) akan diatur menjadi 1, dan program akan melompat ke label SANA. Hal ini berguna untuk menangani perhitungan dalam sistem oktal, di mana angka terbesar hanya 7. Jika nilai lebih kecil dari 8, tidak perlu koreksi tambahan.

Kemudian pada instruksi JC SINI, nilai dalam AL dibandingkan dengan 80H (128 dalam desimal). Jika AL < 80H, maka Carry Flag (CF) = 1, dan program akan melompat ke label SINI untuk melakukan penyesuaian lebih lanjut terhadap hasil pengurangan dalam sistem oktal.

Dan pada intruksi JC yang terakhir digunakan yaitu JC SONO, instruksi ini memastikan bahwa jika hasil pengurangan dalam sistem oktal lebih kecil dari 8, maka program akan melompat ke SONO, di mana register DX dikurangi dengan 800H jika diperlukan.

2. Sama seperti soal no.1 namun menggunakan cara komplemen 8

   Program :

```
01 TITLE PENJUMLAHAN 2 DATA OKTAL 3 DIGIT KOMPLEMEN 8
02 .MODEL SMALL
03 .STACK 100H
04 .DATA
05 .CODE
06 MAIN PROC
07     MOV BX,7777H
08     SUB BX,476H
09     INC BX
10     ADD BX,645H
11     MOV AL,BL
12     AND AL,0FH
13     CMP AL,8
14     JC LUP1
15     ADD BX,8
16 LUP1: MOV AL,BL
17     CMP AL,80H
18     JC LUP2
19     ADD BX,80H
20 LUP2: MOV AL,BH
21     AND AL,0FH
22     CMP AL,08
23     JC LUP3
24     SUB BX,800H
25 LUP3: AND BH,0FH
26     MOV AH,4CH
27     INT 21H
28 MAIN ENDP
29 END MAIN
```

Hasil program :



Analisis data :

Pada soal ke-2, nilai yang dilakukan operasi pengurangan sama dengan pada soal ke-1. Namun pada soal ini operasi pengurangan dilakukan dengan menggunakan metode komplemen 8. Program diawali dengan inisialisasi nilai BX dengan bilangan oktal 7777H, kemudian nilai dalam BX dikurangi dengan 476H untuk mengonversi ke komplemen 8 dan ditambah dengan 1 untuk menyelesaikan proses komplemen 8. Setelah itu, bilangan oktal 645H ditambahkan ke BX untuk mendapatkan hasil penjumlahan.

Selanjutnya, dilakukan koreksi pada setiap digit hasil penjumlahan untuk memastikan bahwa angka yang diperoleh tetap dalam format oktal. Untuk memastikan hasil tetap dalam format oktal yang benar, program melakukan serangkaian koreksi menggunakan operasi logika dan perbandingan. Koreksi ini dilakukan dengan memisahkan digit satuan, puluhan, dan ratusan dari hasil penjumlahan menggunakan instruksi AND untuk mengambil nilai 4 bit terendah dari register AL, serta CMP untuk membandingkannya dengan batas oktal. Jika nilai yang diperiksa lebih besar atau sama dengan 8, maka program akan menyesuaikannya dengan menambahkan atau mengurangi nilai tertentu agar tetap sesuai dengan format oktal.

Instruksi JC (Jump if Carry) digunakan untuk menangani overflow yang terjadi selama proses penjumlahan, memastikan bahwa carry dari satu digit tidak mengganggu digit lainnya. Pada tahap akhir, program menghapus bit yang tidak diperlukan di BH dengan AND.

Akhirnya, program ditutup dengan perintah keluar menggunakan INT 21H. Dengan adanya mekanisme koreksi ini, program dapat menangani carry dan memastikan bahwa hasil penjumlahan tetap dalam format oktal yang benar, sehingga dapat digunakan dalam berbagai operasi bilangan berbasis oktal dalam bahasa Assembly.

3. Buat program untuk mengurangkan data oktal 8 digit berikut dengan data – data oktal lainnya sebagai berikut :

$$74747474 – (7777, 6, 7676, 717273, 123456) = 72473515$$

Pergunakan data area untuk menyimpan data dan output.

Program :

```
TITLE SOAL 3 PENGURANGAN OKTAL 8 DIGIT
.MODEL SMALL
.STACK 100H
.DATA
DAT DW 7474H, 7474H
BIL DW 7777H, 0000H
    DW 7676H, 0006H
    DW 7273H, 0071H
    DW 4567H, 0123H
HSL DW 0,0
.CODE
MAIN PROC
        MOV AX, @DATA
        MOV DS, AX

        MOV AX, DAT
        MOV BX, DAT+2
        MOV DI, 0
LUP0:   SUB AX, [BIL+DI]
        INC DI
        INC DI
        SBB BX, [BIL+DI]

        MOV CL, AL
        AND CL, 0Fh
        CMP CL, 8
        JC LUP1
        SUB AX, 8
        SBB BX, 0

LUP1:   MOV CL, AL
        CMP CL, 80H
        JC LUP2
        SUB AX, 80H
        SBB BX, 0

LUP2:   MOV CL, AH
        AND CL, 0fh
        CMP CL, 8
        JC LUP3
        SUB AX, 800H
        SBB BX, 0

LUP3:   MOV CL, AH
        CMP CL, 80H
        JC LUP4
        SUB AX, 8000H
        SBB BX, 0

LUP4:   MOV CL, BL
        AND CL, 0FH
        CMP CL, 8
        JC LUP5
        SUB BX, 8

LUP5:   MOV CL, BL
        CMP CL, 80H
        JC LUP6
        SUB BX, 80H

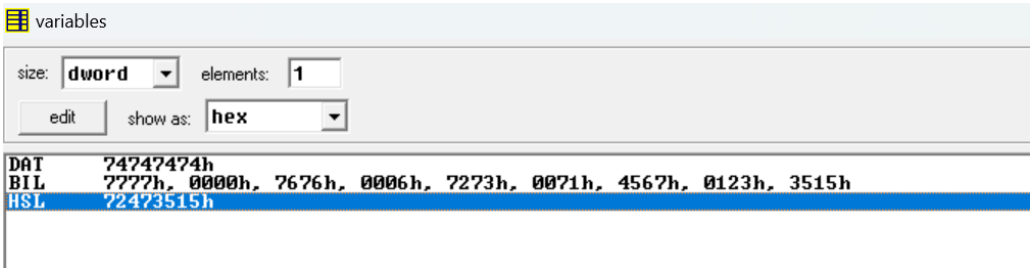
LUP6:   MOV CL, BH
        AND CL, 0FH
        CMP CL, 8
        JC LUP7
        SUB BX, 800H

LUP7:   ADD DI, 2
        CMP DI, 16
        JNZ LUP0
        MOV HSL, AX
        MOV HSL+2, BX

        MOV AH, 4CH
        INT 21H

MAIN    ENDP
        END MAIN
```

Hasil Program :



Analisis Data :

Pada soal ke-3, dibuat program yang melakukan operasi pengurangan pada bilangan oktal 8 digit menggunakan metode komplemen 8. Program ini memanfaatkan instruksi-instruksi dasar Assembly seperti MOV, SUB, SBB, CMP, JC, dan JNZ untuk menjalankan perhitungan dan pengendalian alur program.

Program diawali dengan inisialisasi segmen data, di mana terdapat deklarasi variabel DAT yang berisi angka oktal awal dan BIL yang berisi nilai-nilai yang akan dikurangi dari DAT. Nilai hasil pengurangan disimpan dalam data area dengan variabel HSL. Proses pengurangan dilakukan dengan menggunakan loop yang memproses tiap bagian dari bilangan oktal secara bertahap. Instruksi SUB digunakan untuk mengurangi nilai dari DAT dengan nilai pada BIL, sementara SBB digunakan untuk menangani borrow jika ada.

Instruksi CMP digunakan untuk membandingkan hasil pengurangan dengan batas terakhir agar program dapat menangani carry yang terjadi akibat operasi. Instruksi JC (Jump if Carry) berfungsi untuk mengalihkan eksekusi ke bagian program lain jika terdapat carry. Akhir dari program ini menyimpan hasil akhir ke dalam variabel HSL pada data area, yang dapat dilihat pada tampilan variabel di emulator. Output akhirnya menunjukkan bahwa hasil pengurangan yang tersimpan dalam HSL adalah 72473515h, yang merupakan hasil dari operasi pengurangan bilangan oktal yang telah dilakukan.

4. Buat program untuk melakukan operasi pengurangan berulang oktal 6 digit dengan 2 digit (717273 : 67 = 103310 (10D9h) dengan sisa 34)

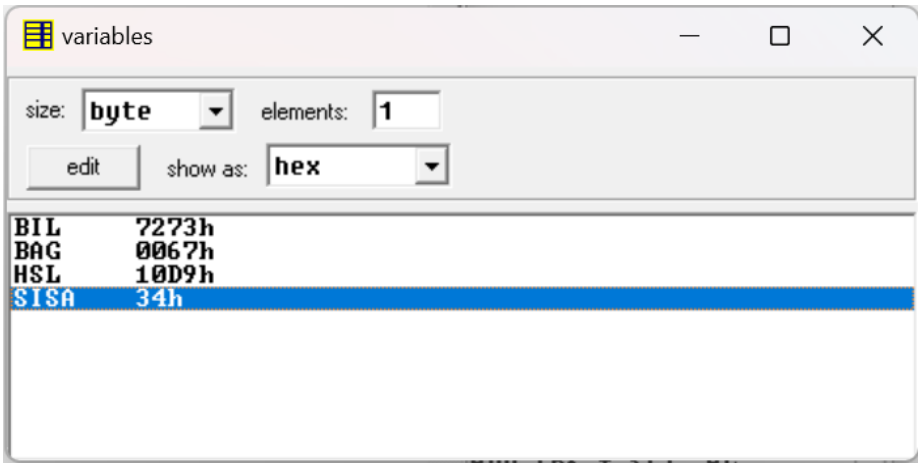
   Program :

```
TITLE PENGURANGAN BERULANG OKTAL SOAL 4
.MODEL SMALL
.STACK 100H
.DATA
     BIL       DW 7273H, 0071H
     BAG       DW 0067H
     HSL       DW 0,0
     SISA      DB 0
.CODE
MAIN PROC
     MOV AX, @DATA
     MOV DS, AX
     MOV DI, 0
     MOV CX, BIL
     MOV BX, BIL+2
     MOV DX, BAG
LPA:
     CMP BX, 0
     JA LPS
     CMP CX, DX
     JAE LPS
     JMP LPD
LPS:
     SUB CX, DX
     SBB BX, 0
     MOV AL, CL
     AND AL, 0FH
     CMP AL, 8
     JC LP1
     SUB CX, 8
     SBB BX, 0
LP1:
     MOV AL, CL
     CMP AL, 80H
     JC LP2
     SUB CX, 80H
     SBB BX, 0

LP2:
     MOV AL, CH
     AND AL, 0FH
     CMP AL, 8
     JC LP3
     SUB CX, 800H
     SBB BX, 0
```

```
47  LP3:
48       MOV AL, CH
49       CMP AL, 80H
50       JC LP4
51       SUB CX, 8000H
52       SBB BX, 0
53
54  LP4:
55       MOV AL, BL
56       AND AL, 0FH
57       CMP AL, 8
58       JC LP5
59       SUB BX, 8
60
61  LP5:
62       MOV AL, BL
63       CMP AL, 80H
64       JC LPU
65       SUB BX, 80H
66  LPU:
67       INC DI
68       JMP LPA
69  LPD:
70       MOV HSL, DI
71       MOV SISA, CL
72
73       MOV AH, 4CH
74       INT 21H
75  MAIN ENDP
76  END MAIN
```

Hasil program :



Analisis data :

Pada soal ke-4, dibuat program untuk melakukan pengurangan berulang pada bilangan oktal 8 digit. Program ini mengurangi dua buah bilangan 8 digit yang terdiri dari pasangan word (masing-masing 16-bit), yaitu BIL sebagai bilangan yang dikurangi dan BAG sebagai pengurang. Pengurangan dilakukan secara berulang hingga nilai BIL tidak lagi cukup untuk dikurangi dengan BAG, baik dari

sisi word high (BX) maupun low (CX). Hasil dari berapa kali pengurangan bisa dilakukan disimpan dalam HSL, sedangkan sisa dari nilai yang tidak bisa dikurangi lagi disimpan dalam SISA.

Langkah pertama dari program ini adalah menginisialisasi segment dan register yang digunakan. Register CX dan BX diisi dengan bilangan awal BIL (masing-masing low dan high word), DX diisi dengan BAG, dan DI sebagai counter pengulangan (hasil akhir). Proses perulangan utama dimulai di label LPA, di mana terdapat dua proses CMP (compare). Pertama, membandingkan nilai BX (high word) dengan nol. Jika BX > 0, maka lompat ke label LPS untuk proses pengurangan. Kedua, jika BX == 0, maka program membandingkan CX (low word) dengan DX (nilai pengurang). Jika CX >= DX, maka lanjut ke proses LPS. Jika kedua kondisi tidak terpenuhi (nilai BIL sudah tidak mencukupi untuk dikurangi), maka program melompat ke LPD untuk menyimpan hasil.

Pada label LPS, program melakukan pengurangan nilai CX dengan DX, dan BX dengan carry dari operasi sebelumnya (menggunakan SBB atau "subtract with borrow"). Setelah pengurangan utama, beberapa langkah koreksi dilakukan untuk menangani bilangan oktal (karena tidak menggunakan basis 10 maupun 16 sepenuhnya). Instruksi seperti AND, CMP, JC, SUB, dan SBB digunakan untuk mengoreksi digit-digit jika hasil pengurangan melebihi batas representasi oktal, seperti > 7 atau bit tanda 80H. Koreksi ini dilakukan pada masing-masing byte dari word bilangan, dimulai dari CL, CH, BL, hingga BH.

Setiap kali pengurangan berhasil dilakukan, nilai DI (counter pengurangan) akan di-increment di LPU, kemudian proses kembali ke LPA untuk mengecek apakah bisa dilakukan pengurangan berikutnya. Jika tidak bisa, maka program melompat ke LPD, di mana DI disimpan ke variabel HSL sebagai jumlah pengurangan yang berhasil dilakukan, dan nilai sisa pengurangan di CL disimpan ke SISA. Program kemudian berhenti dengan INT 21H.

Hasil akhir pada tampilan menunjukkan bahwa HSL = 10D9h, yang berarti proses pengurangan dilakukan sebanyak 10D9h kali, dan sisa terakhir yang tidak bisa dikurangi adalah 34h (dalam SISA). Ini menunjukkan bahwa program berhasil melakukan pengurangan oktal 8 digit secara berulang hingga tidak bisa lagi dikurangi, sambil tetap mempertahankan validitas digit oktal melalui langkah-langkah koreksi bit.

F. KESIMPULAN

Dari hasil praktikum dan percobaan dengan menyelesaikan empat soal diatas, dapat disimpulkan bahwa operasi pengurangan dan pembagian 1dengan menggunakan metode pengurangan berulang bilangan oktal dalam format BCD pada bahasa assembly dengan EMU8086 dapat dilakukan dengan menambahkan faktor koreksi bilangan oktal agar hasilnya tetap sesuai dalam format bilangan oktal. Penggunaan register dan data area harus di sesuaikan dengan data yang akan dijumlahkan, maka dari itu praktikan dapat lebih memahami bagaimana fungsi register dan data area pada operasi-operasi bilangan yang dilakukan pada software EMU8086.

DAFTAR PUSTAKA

Hasanah, Rahmawati. 2019. Mikroprosesor dan Interface. Bandung: Politeknik Negeri Bandung

S.to. 2001. “Pemrograman Dengan Bahasa Assembly Edisi Online Versi 1.0